

Documentación y pruebas de la api Apartamentos

Universidad Cristóbal Colón
Estudiante: Erick Gabriel Hernández Rebolledo
Docente: Jesús Leonardo López Hernández
Materia: Arquitectura Orientada A Servicios
Fecha: 14-10-2026

1. Enlace al repositorio

El código completo del proyecto está disponible en el siguiente repositorio de GitHub:

https://github.com/ErickGHR/apartamentos-api

El repositorio contiene el código fuente, las instrucciones de ejecución, las pruebas automáticas y la colección de Postman utilizada para comprobar el funcionamiento de la API.

2. Descripción del proyecto

Este proyecto consiste en una API REST para administrar un servicio de renta de apartamentos.

La aplicación permite registrar propietarios y huéspedes, dar de alta propiedades, agregar unidades rentables, crear reservaciones y llevar el control de los pagos. También cuenta con funciones relacionadas con disponibilidad, reseñas, imágenes y notificaciones.

Además de la API, el proyecto incluye algunas páginas desarrolladas con Thymeleaf para consultar y administrar información desde el navegador.

El código está dividido en modelos, repositorios, servicios y controladores. Esta organización permite mantener separadas las entidades, las consultas a la base de datos, las reglas del sistema y los endpoints.

3. Tecnologías utilizadas

Para desarrollar y probar el proyecto utilicé:

- Java 21.
- Spring Boot.
- Spring Web.
- Spring Data JPA.
- Thymeleaf.
- MySQL.
- Maven.
- Postman.
- Git y GitHub.

4. Funciones principales

Las funciones principales del sistema son:

- Registro de clientes propietarios.
- Registro de clientes huéspedes.
- Administración de propiedades.
- Registro de unidades rentables.
- Control de disponibilidad.

- Creación de reservaciones.
- Registro y confirmación de pagos.
- Registro de reseñas.
- Administración de imágenes y notificaciones.
- Búsqueda de propiedades por tipo.
- Búsqueda de propiedades por propietario.

Las rutas principales son:

/api/clientes

/api/propiedades

/api/unidades

/api/disponibilidades

/api/reservas

/api/pagos

/api/resenas

/api/notificaciones

/api/propiedad-imágenes

Y también hay algunas consultas:

GET /api/propiedades/tipo/{tipo}

GET /api/propiedades/propietario/{idPropietario}

5. Instrucciones para ejecutar el proyecto

Para ejecutar el proyecto se necesita tener instalado Java 21, Git y MySQL.

Primero se descarga el código desde GitHub:

git clone https://github.com/ErickGHR/apartamentos-api.git

cd apartamentos-api

Después se ejecuta el script incluido:

.\run-local.ps1

El script solicita la contraseña local de MySQL. La aplicación crea automáticamente la base de datos apartamentos cuando el usuario cuenta con los permisos necesarios.

Cuando el proyecto termina de iniciar, queda disponible en:

http://localhost:8000

La misma dirección se puede abrir desde un navegador para consultar la interfaz web del proyecto.

Para detener la aplicación se presiona **Ctrl + C** en la terminal.

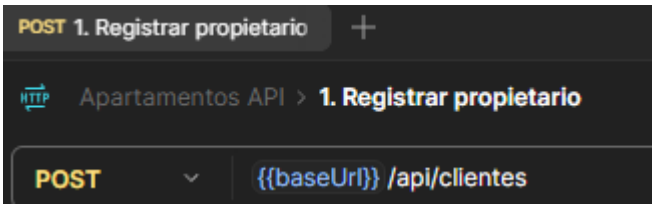
6. Pruebas realizadas en Postman

Para comprobar el funcionamiento de la API importé la colección incluida en el repositorio. Las peticiones se ejecutaron en orden porque cada una utiliza los identificadores generados en las pruebas anteriores.

6.1 Registro de un propietario

En la primera prueba registré un cliente con el tipo PROPIETARIO. Envié su nombre, apellidos, correo electrónico, teléfono, fecha de nacimiento, INE, dirección y contraseña.

La API respondió con 201 Created y generó automáticamente el identificador del propietario. La contraseña fue recibida por la aplicación, pero no apareció en la respuesta por seguridad.



Body Cookies Headers 5 Test Results 0/1

{ } JSON Preview Visualize


```
{
    "apellidos": "López",
    "direccion": "Calle Hidalgo 100",
    "email": "ana1792007550@example.com",
    "estatus": "ACTIVO",
    "fechaNacimiento": "1990-06-15",
    "fechaRegistro": "2026-10-14T13:52:30.3366804",
    "id": 1,
    "ine": "INE-P-1792007550",
    "nombres": "Ana",
    "telefono": "8112345678",
    "tipo": "PROPIETARIO"
}
```


Body Cookies Headers 5 Test Results 0/1

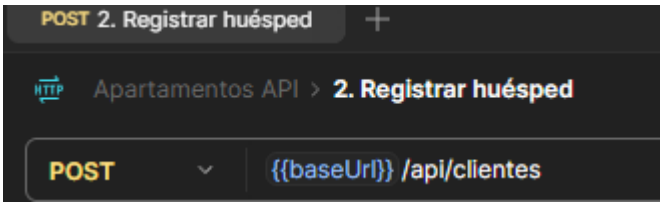
{ } JSON Preview Visualize

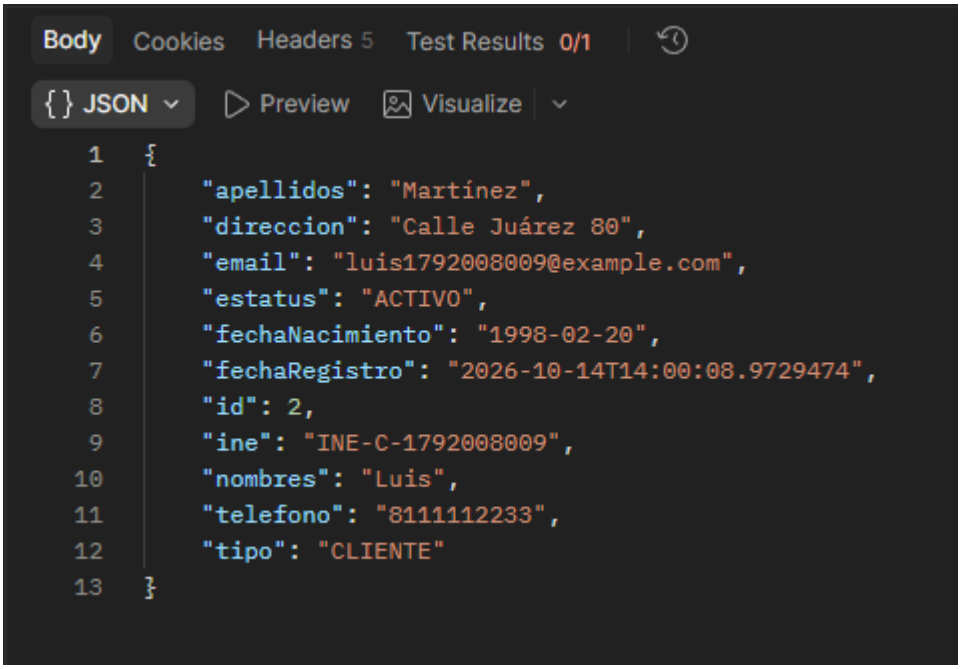

```
{
    "apellidos": "Martínez",
    "direccion": "Calle Juárez 80",
    "email": "luis1792008009@example.com",
    "estatus": "ACTIVO",
    "fechaNacimiento": "1998-02-20",
    "fechaRegistro": "2026-10-14T14:00:08.9729474",
    "id": 2,
    "ine": "INE-C-1792008009",
    "nombres": "Luis",
    "telefono": "8111112233",
    "tipo": "CLIENTE"
}
```

6.2 Registro de un huésped

En la segunda petición registré un cliente con el tipo CLIENTE, que representa a la persona que puede realizar una reservación.

La respuesta obtenida fue 201 Created, indicando que el huésped se guardó correctamente.



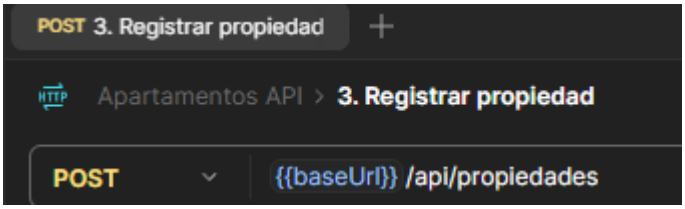


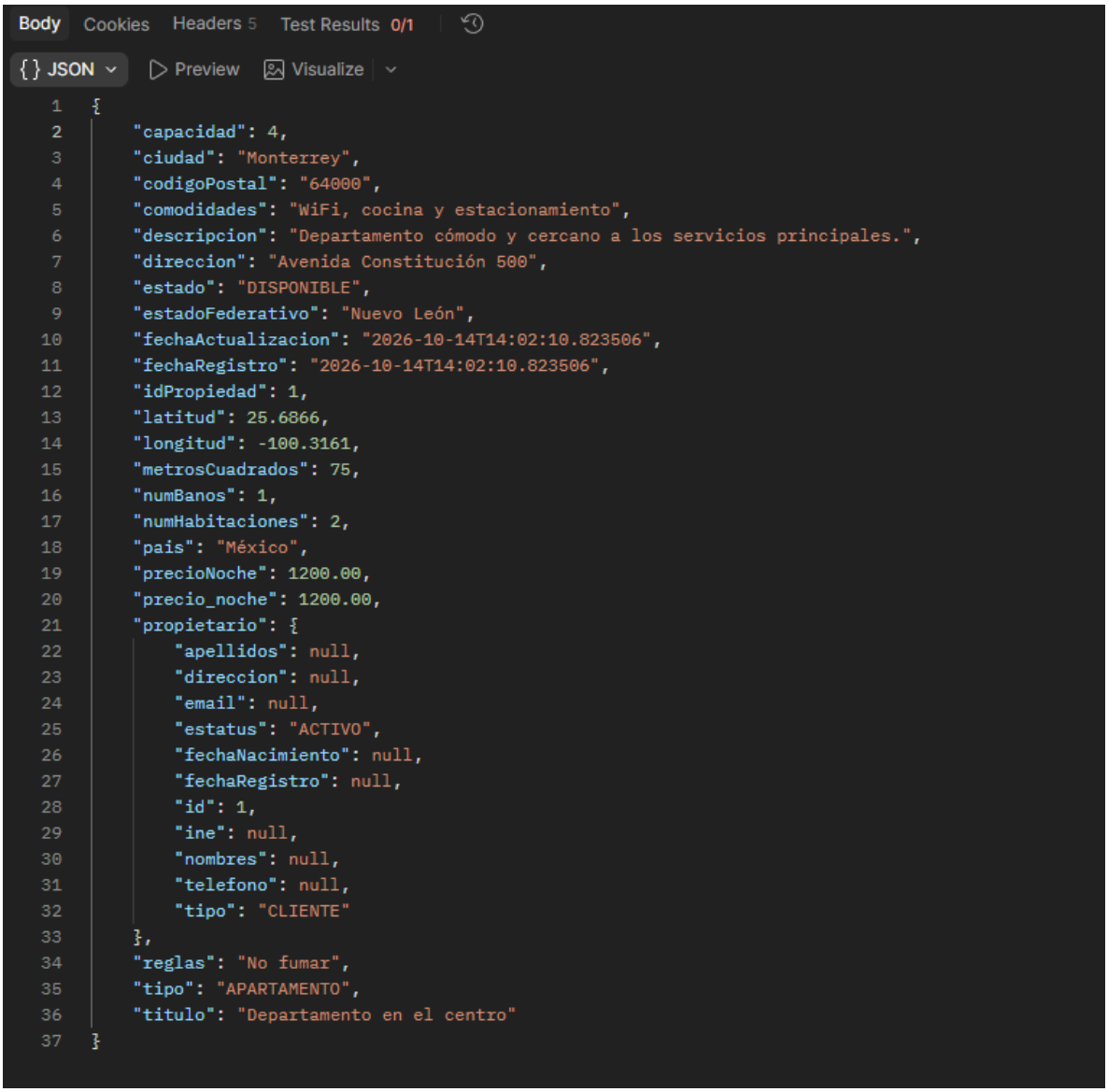
6.3 Registro de una propiedad

Después registré una propiedad y la relacioné con el propietario creado anteriormente.

En esta petición envié el tipo de propiedad, título, descripción, dirección, ciudad, estado, código postal, país, ubicación, precio por noche, capacidad y características generales.

La API respondió con 201 Created.

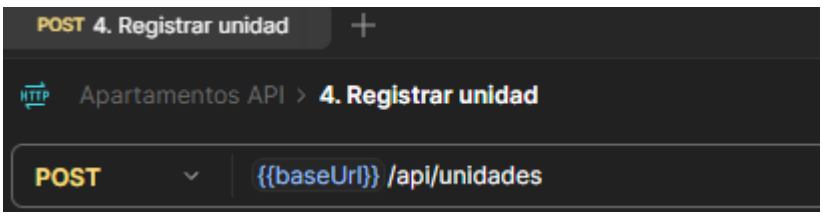


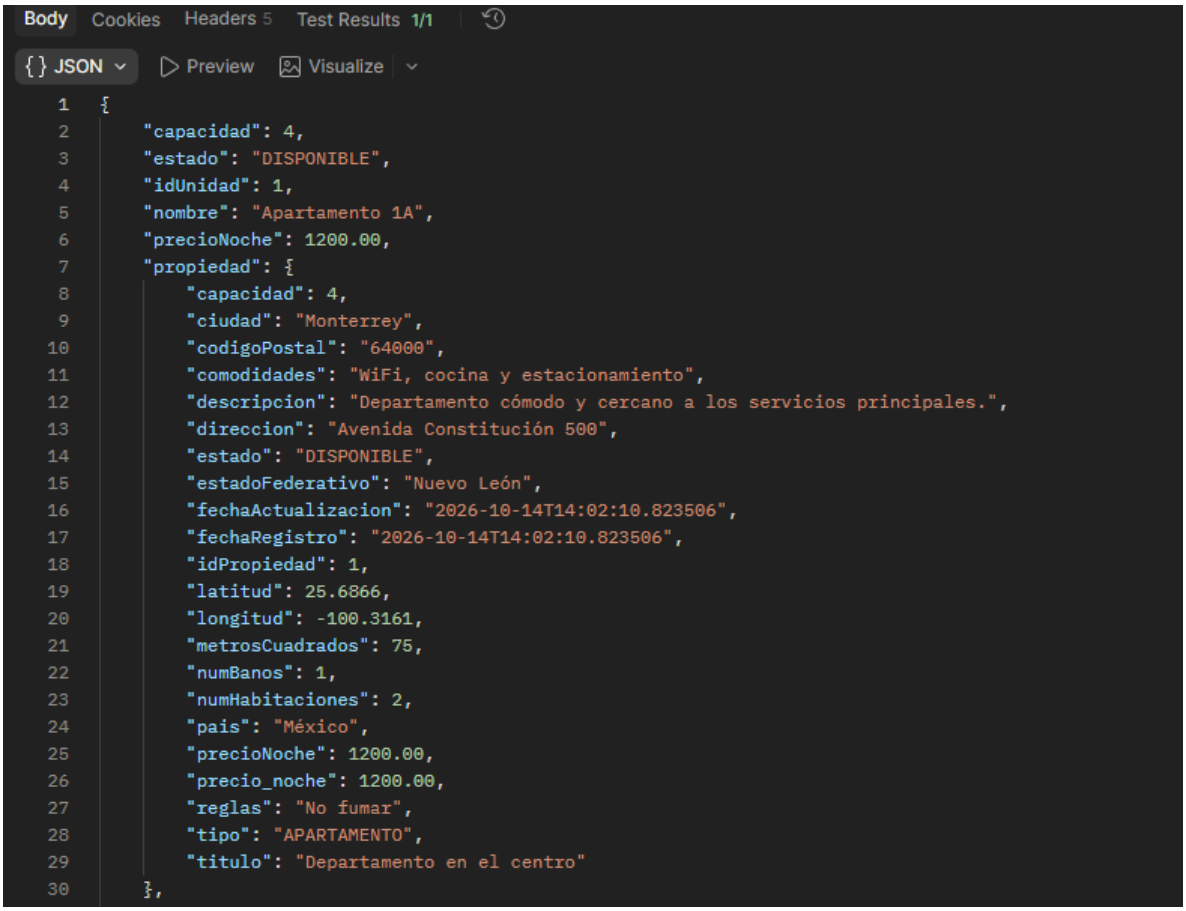


6.4 Registro de una unidad rentable

En esta prueba agregué una unidad a la propiedad. La unidad representa el espacio específico que puede reservarse.

Se enviaron el nombre, tipo, capacidad, precio por noche y estado de disponibilidad. La respuesta fue 201 Created.

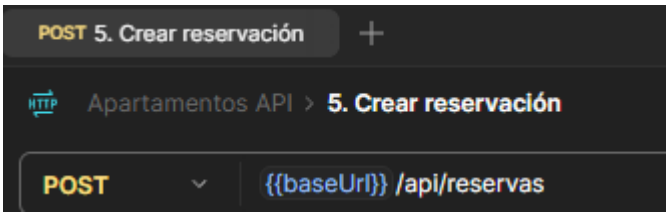


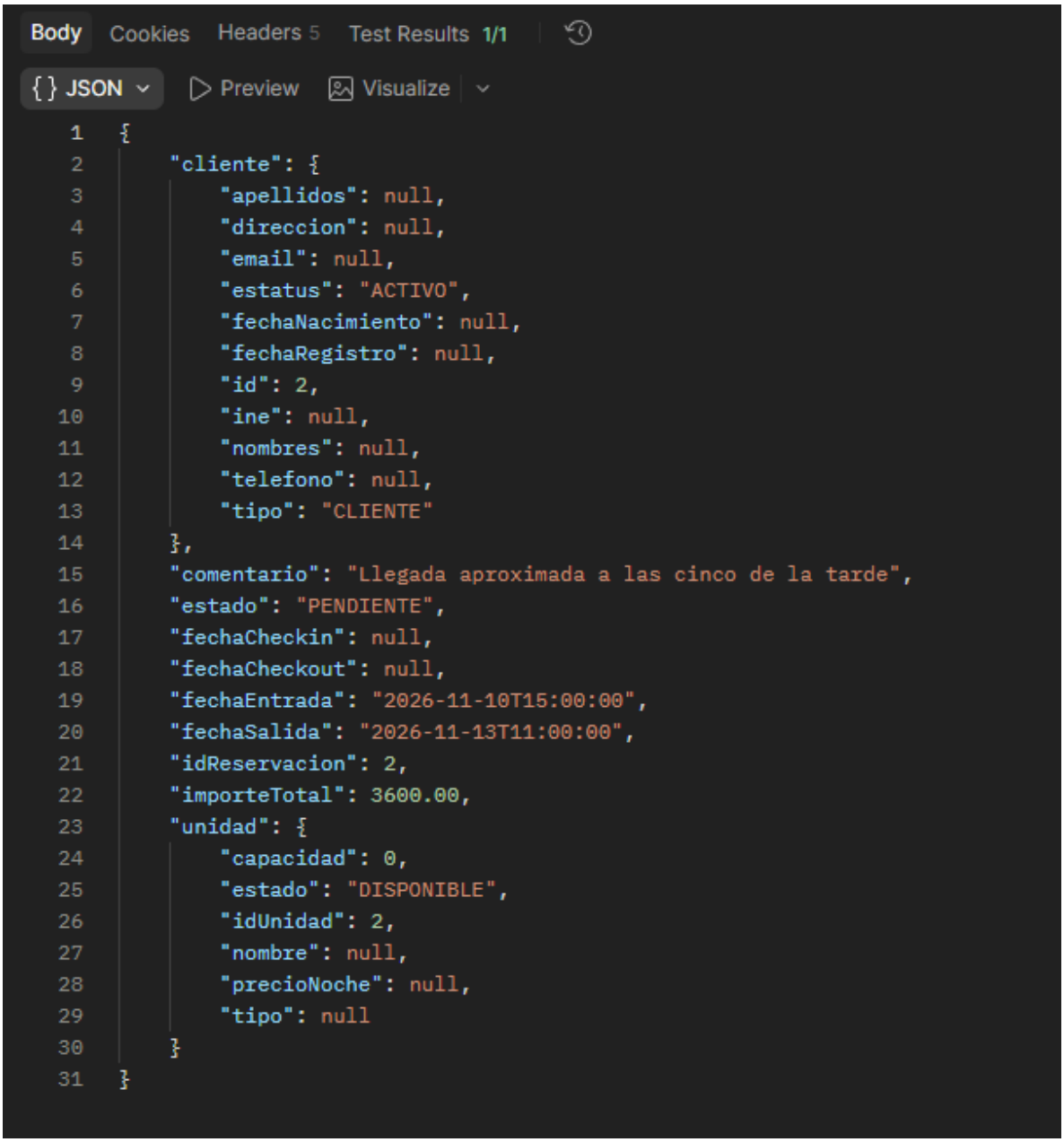


6.5 Creación de una reservación

Para crear la reservación relacioné al huésped con la unidad registrada. También envié las fechas de entrada y salida, el importe y un comentario.

La API revisó que las fechas fueran correctas y que no existiera otra reservación cruzada para la misma unidad. La respuesta obtenida fue 201 Created.



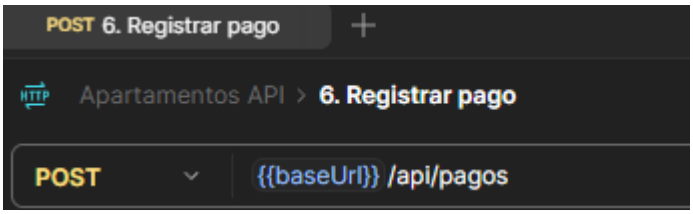


6.6 Registro de un pago

En la sexta prueba registré un pago relacionado con la reservación.

La petición incluyó el importe, la fecha de vencimiento, el método de pago y el estado inicial. La aplicación relacionó automáticamente el pago con el cliente de la reservación y generó una referencia de pago.

La respuesta fue 201 Created.



Body Cookies Headers 5 Test Results 1/1

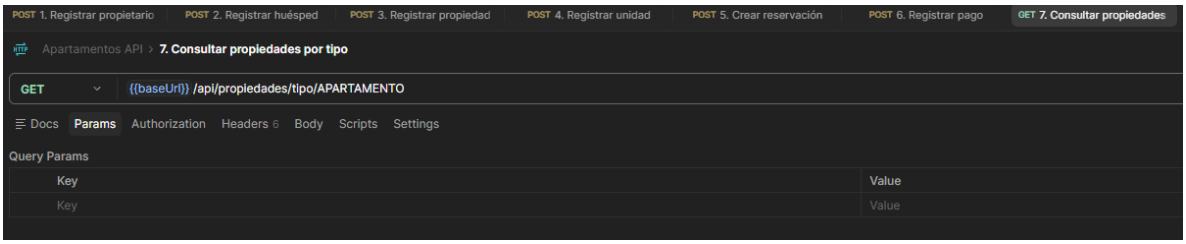
{ } JSON Preview Visualize

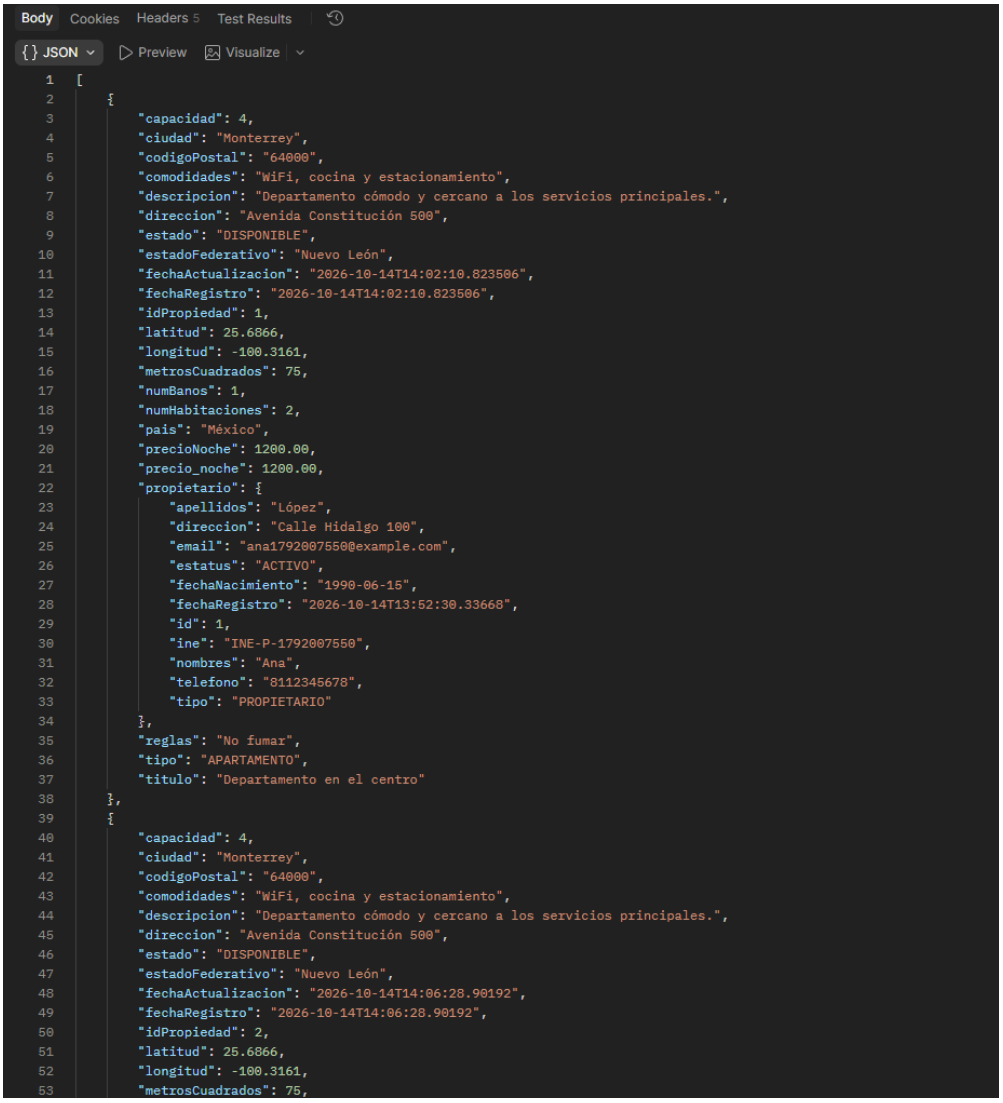

```
{
    "cliente": {
        "apellidos": "Martínez",
        "direccion": "Calle Juárez 80",
        "email": "luis1792008009@example.com",
        "estatus": "ACTIVO",
        "fechaNacimiento": "1998-02-20",
        "fechaRegistro": "2026-10-14T14:00:08.972947",
        "id": 2,
        "ine": "INE-C-1792008009",
        "nombres": "Luis",
        "telefono": "8111112233",
        "tipo": "CLIENTE"
    },
    "createdAt": "2026-10-14T14:06:56.1778284",
    "estado": "PENDIENTE",
    "fechaPago": null,
    "fechaVencimiento": "2026-11-05",
    "idPago": 2,
    "importe": 3600.00,
    "metodo": "TRANSFERENCIA",
    "referenciaPago": "PAG-DAA9AB1FA53D4440",
    "reserva": {
        "comentario": "Llegada aproximada a las cinco de la tarde",
        "estado": "PENDIENTE",
        "fechaCheckin": null,
        "fechaCheckout": null,
        "fechaEntrada": "2026-11-10T15:00:00",
        "fechaSalida": "2026-11-13T11:00:00",
        "idReservacion": 2,
        "importeTotal": 3600.00
    }
}
```

6.7 Consulta de propiedades por tipo

En esta prueba realicé una petición GET para buscar las propiedades del tipo APARTAMENTO.

La API respondió con 200 OK y mostró la propiedad registrada previamente.



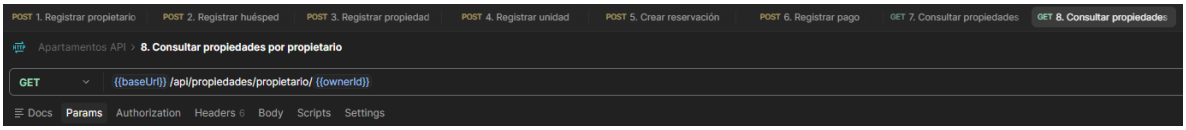


```
        "metrosCuadrados": 75,
        "numBanos": 1,
        "numHabitaciones": 2,
        "pais": "México",
        "precioNoche": 1200.00,
        "precio_noche": 1200.00,
        "propietario": {
            "apellidos": "López",
            "direccion": "Calle Hidalgo 100",
            "email": "ana1792007550@example.com",
            "estatus": "ACTIVO",
            "fechaNacimiento": "1990-06-15",
            "fechaRegistro": "2026-10-14T13:52:30.33668",
            "id": 1,
            "ine": "INE-P-1792007550",
            "nombres": "Ana",
            "telefono": "8112345678",
            "tipo": "PROPIETARIO"
        },
        "reglas": "No fumar",
        "tipo": "APARTAMENTO",
        "titulo": "Departamento en el centro"
    }
]
```


6.8 Consulta de propiedades por propietario

Después consulté las propiedades relacionadas con el propietario registrado en la primera prueba.

La respuesta fue 200 OK y mostró la propiedad correspondiente al identificador enviado.



Body Cookies Headers 5 Test Results

{ } JSON Preview Visualize

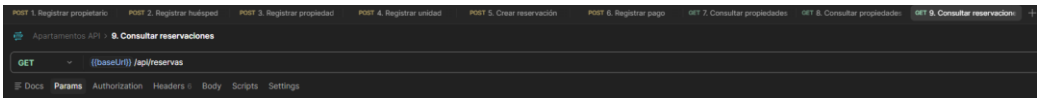

```
[
    {
        "capacidad": 4,
        "ciudad": "Monterrey",
        "codigoPostal": "64000",
        "comodidades": "WiFi, cocina y estacionamiento",
        "descripcion": "Departamento cómodo y cercano a los servicios principales.",
        "direccion": "Avenida Constitución 500",
        "estado": "DISPONIBLE",
        "estadoFederativo": "Nuevo León",
        "fechaActualizacion": "2026-10-14T14:02:10.823506",
        "fechaRegistro": "2026-10-14T14:02:10.823506",
        "idPropiedad": 1,
        "latitud": 25.6866,
        "longitud": -100.3161,
        "metrosCuadrados": 75,
        "numBanos": 1,
        "numHabitaciones": 2,
        "pais": "México",
        "precioNoche": 1200.00,
        "precio_noche": 1200.00,
        "propietario": {
            "apellidos": "López",
            "direccion": "Calle Hidalgo 100",
            "email": "ana1792007550@example.com",
            "estatus": "ACTIVO",
            "fechaNacimiento": "1990-06-15",
            "fechaRegistro": "2026-10-14T13:52:30.33668",
            "id": 1,
            "ine": "INE-P-1792007550",
            "nombres": "Ana",
            "telefono": "8112345678",
            "tipo": "PROPIETARIO"
        },
        "reglas": "No fumar",
        "tipo": "APARTAMENTO",
        "titulo": "Departamento en el centro"
    },
    {
        "capacidad": 4,
        "ciudad": "Monterrey",
        "codigoPostal": "64000",
        "comodidades": "WiFi, cocina y estacionamiento",
        "descripcion": "Departamento cómodo y cercano a los servicios principales.",
        "direccion": "Avenida Constitución 500",
        "estado": "DISPONIBLE",
        "estadoFederativo": "Nuevo León",
        "fechaActualizacion": "2026-10-14T14:06:28.90192",
        "fechaRegistro": "2026-10-14T14:06:28.90192",
        "idPropiedad": 2,
        "latitud": 25.6866,
        "longitud": -100.3161,
        "metrosCuadrados": 75,
```

```
        "numBanos": 1,
        "numHabitaciones": 2,
        "pais": "México",
        "precioNoche": 1200.00,
        "precio_noche": 1200.00,
        "propietario": {
            "apellidos": "López",
            "direccion": "Calle Hidalgo 100",
            "email": "ana1792007550@example.com",
            "estatus": "ACTIVO",
            "fechaNacimiento": "1990-06-15",
            "fechaRegistro": "2026-10-14T13:52:30.33668",
            "id": 1,
            "ine": "INE-P-1792007550",
            "nombres": "Ana",
            "telefono": "8112345678",
            "tipo": "PROPIETARIO"
        },
        "reglas": "No fumar",
        "tipo": "APARTAMENTO",
        "titulo": "Departamento en el centro"
    }
]
```

6.9 Consulta de reservaciones

Finalmente realicé una petición GET para consultar las reservaciones almacenadas.

La API respondió con 200 OK y mostró la unidad, el cliente, las fechas, el importe total, el estado y el comentario de la reservación.





7. Resultado de las pruebas

Las nueve peticiones de Postman se ejecutaron correctamente.

Durante las pruebas se comprobó que los clientes, propiedades, unidades, reservaciones y pagos se almacenan en MySQL. También se verificó que las relaciones entre los registros funcionan y que las búsquedas por tipo y propietario devuelven la información esperada.

Los identificadores generados en cada petición se guardaron automáticamente dentro de la colección de Postman para utilizarlos en las siguientes pruebas.

8. Conclusión

Este proyecto permitió desarrollar una API REST para controlar un servicio de renta de apartamentos.

La aplicación puede administrar la información de propietarios, huéspedes, propiedades, unidades, reservaciones y pagos. También cuenta con validaciones para evitar datos incompletos, fechas incorrectas y reservaciones cruzadas.

Las pruebas realizadas en Postman permitieron comprobar que los endpoints responden correctamente y que la información se guarda y consulta de acuerdo con lo esperado.